

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.

Task -1: Create Project Structure as follows:

OSSP/

```
|
|
|— .git/          # Git repository (created automatically)
|— .gitignore     # Files/folders Git should ignore
|— README.md      # Project description and instructions
|— LICENSE        # Project license (optional)
|— Makefile       # Build instructions
|
|— src/           # Source code
|   |— main.c
|   |— shell.c
|   |— parser.c
|   |— executor.c
|   └─ builtin.c
|
|— include/       # Header files
|   |— shell.h
|   |— parser.h
|   |— executor.h
|   └─ builtin.h
|
```

```
└─ obj/          # Object files (.o) (generated)
|
└─ bin/          # Executable file (generated)
|   └─ my_shell
|
└─ docs/         # Documentation
|   └─ design.md
|   └─ report.pdf
|
└─ tests/        # Test programs
|   └─ test_parser.c
|   └─ test_executor.c
|
└─ scripts/      # Helper scripts
|   └─ run.sh
|
└─ assets/       # Images/screenshots
    └─ architecture.png
```

Note: For creating this structure use mkdir and touch commands.

```
saicharan_reddi@SAICHARAN x + v
root@Mani: mkdir OSSP
cd OSSP
mkdir: OSSP: File exists
-bash: cd: OSSP: Not a directory
root@Mani: mkdir src include obj bin docs tests scripts assets
root@Mani: touch .gitignore README.md LICENSE Makefile
touch src/main.c src/shell.c src/parser.c src/executor.c src/builtin.c
touch include/shell.h include/parser.h include/executor.h include/builtin.h
touch docs/design.md docs/report.pdf
touch tests/test_parser.c tests/test_executor.c
touch scripts/run.sh
touch assets/architecture.png
root@Mani: git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/saicharan_reddi/.git/
root@Mani: ls -R
.:
BufferSize.c.save  ForkProgram.c  PROCESSTASK.save  SystemCallParent.c  filecopy.c      scripts      tests
DisplayPID         LICENSE        README.md         assets              fliecopy.c.save  shellcommand
DisplayPID.c       Makefile       SystemCall        bin                include          shellcommand.c
FileManagement     OSSP           SystemCall.c      copyfile.c          kill.c           snap
ForkProgram        OS_LAB         SystemCallParent  docs               obj              src

./FileManagement:
FileManagement

./FileManagement/FileManagement:

./OS_LAB:
```

```

./OS_LAB:
task1 task1.c

./assets:
architecture.png

./bin:

./docs:
design.md report.pdf

./include:
builtin.h executor.h parser.h shell.h

./obj:

./scripts:
run.sh

./snap:
tree

./snap/tree:
54 common current

./snap/tree/54:

./snap/tree/common:
marker_publisher_changed_announcement_shown

./src:
builtin.c executor.c main.c parser.c shell.c

./tests:
test_executor.c test_parser.c

```

Report:

The source files in src, header files in include, object files in obj, executable files in bin, documentation in docs, tests in tests, scripts in scripts, and images in assets. mkdir OSSP cd OSSP mkdir src include obj bin docs tests scripts assets touch .gitignore README.md LICENSE Makefile touch src/main.c src/shell.c src/parser.c src/executor.c src/builtin.c touch include/shell.h include/parser.h include/executor.h include/builtin.h touch docs/design.md docs/report.pdf touch tests/test_parser.c tests/test_executor.c touch scripts/run.sh touch assets/architecture.png git init ls -R. The OSSP project structure was successfully created and a Git repository was initialized. The Linux environment was successfully prepared for developing the Operating System Shell Project using GCC, Git, and Make.

2. To Analyze process abstraction, execute `fork()`, understand `exec()` family, analyze parent-child relationships, inspect process tree, practice system call tracing.

Task-1: Using a manual command understand the `fork()` and `exec()` system calls

```
saicharan_reddi@SAICHARAN x + v
fork(2) System Calls Manual fork(2)

NAME
fork - create a child process

LIBRARY
Standard C library (libc, -lc)

SYNOPSIS
#include <unistd.h>

pid_t fork(void);

DESCRIPTION
fork() creates a new process by duplicating the calling process. The new process is referred to as the child process. The calling process is referred to as the parent process.

The child process and the parent process run in separate memory spaces. At the time of fork() both memory spaces have the same content. Memory writes, file mappings (mmap(2)), and unmappings (munmap(2)) performed by one of the processes do not affect the other.

The child process is an exact duplicate of the parent process except for the following points:



- The child has its own unique process ID, and this PID does not match the ID of any existing process group (setpgid(2)) or session.
- The child's parent process ID is the same as the parent's process ID.
- The child does not inherit its parent's memory locks (mlock(2), mlockall(2)).
- Process resource utilizations (getrusage(2)) and CPU time counters (times(2)) are reset to zero in the child.
- The child's set of pending signals is initially empty (sigpending(2)).
- The child does not inherit semaphore adjustments from its parent (semop(2)).
- The child does not inherit process-associated record locks from its parent (fcntl(2)). (On the other hand, it does inherit fcntl(2) open file description locks and flock(2) locks from its parent.)


- The child does not inherit timers from its parent (setitimer(2), alarm(2), timer_create(2)).
- The child does not inherit outstanding asynchronous I/O operations from its parent (aio_read(3), aio_write(3)), nor does it inherit any asynchronous I/O contexts from its parent (see io_setup(2)).



The process attributes in the preceding list are all specified in POSIX.1. The parent and child also differ with respect to the following Linux-specific process attributes:



- The child does not inherit directory change notifications (dnotify) from its parent (see the description of F_NOTIFY in fcntl(2)).
- The prctl(2) PR_SET_PDEATHSIG setting is reset so that the child does not receive a signal when its parent terminates.
- The default timer slack value is set to the parent's current timer slack value. See the description of PR_SET_TIMERSLACK in prctl(2).
- Memory mappings that have been marked with the madvise(2) MADV_DONTFORK flag are not inherited across a fork().
- Memory in address ranges that have been marked with the madvise(2) MADV_WIPEONFORK flag is zeroed in the child after a fork(). (The MADV_WIPEONFORK setting remains in place for those address ranges in the child.)
- The termination signal of the child is always SIGCHLD (see clone(2)).
- The port access permission bits set by ioperm(2) are not inherited by the child; the child must turn on any bits that it requires using ioperm(2).

```

RETURN VALUE

On success, the PID of the child process is returned in the parent, and 0 is returned in the child. On failure, -1 is returned in the parent, no child process is created, and `errno` is set to indicate the error.

ERRORS

EAGAIN A system-imposed limit on the number of threads was encountered. There are a number of limits that may trigger this error:

- the `RLIMIT_NPROC` soft resource limit (set via `setrlimit(2)`), which limits the number of processes and threads for a real user ID, was reached;
- the kernel's system-wide limit on the number of processes and threads, `/proc/sys/kernel/threads-max`, was reached (see `proc(5)`);
- the maximum number of PIDs, `/proc/sys/kernel/pid_max`, was reached (see `proc(5)`); or
- the PID limit (`pids_max`) imposed by the cgroup "process number" (PIDs) controller was reached.

EAGAIN The caller is operating under the `SCHED_DEADLINE` scheduling policy and does not have the `reset-on-fork` flag set. See `sched(7)`.

ENOMEM `fork()` failed to allocate the necessary kernel structures because memory is tight.

ENOMEM An attempt was made to create a child process in a PID namespace whose "init" process has terminated. See `pid_namespaces(7)`.

ENOSYS `fork()` is not supported on this platform (for example, hardware without a Memory-Management Unit).

ERESTARTNOINTR (since Linux 2.6.17)

System call was interrupted by a signal and will be restarted. (This can be seen only during a trace.)

VERSIONS**C library/kernel differences**

Since glibc 2.3.3, rather than invoking the kernel's `fork()` system call, the glibc `fork()` wrapper that is provided as part of the NPTL threading implementation invokes `clone(2)` with flags that provide the same effect as the traditional system call. (A call to `fork()` is equivalent to a call to `clone(2)` specifying flags as just `SIGCHLD`.) The glibc wrapper invokes any fork handlers that have been established using `pthread_atfork(3)`.

Async-signal safety

`_Fork(3)` is an async-signal safe variant of `fork(2)`.

STANDARDS

POSIX.1-2024.

HISTORY

4.3BSD, SVr4, POSIX.1-1988.

NOTES

Under Linux, `fork()` is implemented using copy-on-write pages, so the only penalty that it incurs is the time and memory required to duplicate the parent's page tables, and to create a unique task structure for the child.

EXAMPLES

See `pipe(2)` and `wait(2)` for more examples.

```
saicharan_reddi@SAICHARAN x + v
EXAMPLES
    See pipe(2) and wait(2) for more examples.

#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

int
main(void)
{
    pid_t pid;

    if (signal(SIGCHLD, SIG_IGN) == SIG_ERR) {
        perror("signal");
        exit(EXIT_FAILURE);
    }
    pid = fork();
    switch (pid) {
        case -1:
            perror("fork");
            exit(EXIT_FAILURE);
        case 0:
            puts("Child exiting.");
            fflush(stdout);
            _exit(EXIT_SUCCESS);
        default:
            printf("Child is PID %jd\n", (intmax_t) pid);
            puts("Parent exiting.");
            exit(EXIT_SUCCESS);
    }
}

SEE ALSO
    clone(2), execve(2), exit(2), _exit(2), setrlimit(2), unshare(2), vfork(2), wait(2), daemon(3), _Fork(3),
    pthread_atfork(3), capabilities(7), credentials(7)

Linux man-pages 6.17                                2026-02-08                                fork(2)
```

```
saicharan_reddi@SAICHARAN x + v
xec(3) Library Functions Manual xec(3)
NAME
    execl, execlp, execlx, execv, execvp, execvpe - execute a file
LIBRARY
    Standard C library (libc, -lc)
SYNOPSIS
#include <unistd.h>

extern char **environ;

int execl(const char *path, const char *arg, ...
/*, (char *) NULL */);
int execlp(const char *file, const char *arg, ...
/*, (char *) NULL */);
int execlx(const char *path, const char *arg, ...
/*, (char *) NULL, char *const envp[] */);
int execv(const char *path, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execvpe(const char *file, char *const argv[], char *const envp[]);

Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

execvpe():
    _GNU_SOURCE
DESCRIPTION
    The exec() family of functions replaces the current process image with a new process image. The functions described in this manual page are layered
    on top of execve(2). (See the manual page for execve(2) for further details about the replacement of the current process image.)

    The initial argument for these functions is the name of a file that is to be executed.

    The functions can be grouped based on the letters following the "exec" prefix.

1 - execl(), execlp(), execlx()
    The const char *arg and subsequent ellipses can be thought of as arg0, arg1, ..., argn. Together they describe a list of one or more pointers to
    null-terminated strings that represent the argument list available to the executed program. The first argument, by convention, should point to the
    filename associated with the file being executed. The list of arguments must be terminated by a null pointer, and, since these are variadic func-
```

v - `execv()`, `execvp()`, `execvpe()`
The `char *const argv[]` argument is an array of pointers to null-terminated strings that represent the argument list available to the new process. The first argument, by convention, should point to the filename associated with the file being executed. The array of pointers *must* be terminated by a null pointer.

e - `execle()`, `execvpe()`
The environment of the new process image is specified via the argument `envp`. The `envp` argument is an array of pointers to null-terminated strings and *must* be terminated by a null pointer.

All other `exec()` functions (which do not include 'e' in the suffix) take the environment for the new process image from the external variable `environ` in the calling process.

p - `execlp()`, `execvp()`, `execvpe()`
These functions duplicate the actions of the shell in searching for an executable file if the specified filename does not contain a slash (/) character. The file is sought in the colon-separated list of directory pathnames specified in the `PATH` environment variable. If this variable is defined, the path list defaults to a list that includes the directories returned by `confstr(_CS_PATH)` (which typically returns the "/bin:/usr/bin") and possibly also the current working directory; see `VERSIONS` for further details.

`execvpe()` searches for the program using the value of `PATH` from the caller's environment, not from the `envp` argument.

If the specified filename includes a slash character, then `PATH` is ignored, and the file at the specified pathname is executed.

In addition, certain errors are treated specially.

If permission is denied for a file (the attempted `execve(2)` failed with the error `EACCES`), these functions will continue searching the rest of the search path. If no other file is found, however, they will return with `errno` set to `EACCES`.

If the header of a file isn't recognized (the attempted `execve(2)` failed with the error `ENOEXEC`), these functions will execute the shell (`/bin/sh`) with the path of the file as its first argument. (If this attempt fails, no further searching is done.)

All other `exec()` functions (which do not include 'p' in the suffix) take as their first argument a (relative or absolute) pathname that identifies the program to be executed.

RETURN VALUE
The `exec()` functions return only if an error has occurred. The return value is -1, and `errno` is set to indicate the error.

ERRORS
All of these functions may fail and set `errno` for any of the errors specified for `execve(2)`.

ATTRIBUTES
For an explanation of the terms used in this section, see `attributes(7)`.

Interface	Attribute	Value
<code>execl()</code> , <code>execle()</code> , <code>execv()</code>	Thread safety	MT-Safe
<code>execlp()</code> , <code>execvp()</code> , <code>execvpe()</code>	Thread safety	MT-Safe env

VERSIONS

STANDARDS
`environ`
`execl()`
`execlp()`
`execle()`
`execv()`
`execvp()`
POSIX.1-2008.

`execvpe()`
GNU.

HISTORY
`environ`
`execl()`
`execlp()`
`execle()`
`execv()`
`execvp()`
POSIX.1-2001.

`execvpe()`
glibc 2.11.

BUGS
Before glibc 2.24, `execl()` and `execle()` employed `realloc(3)` internally and were consequently not `async-signal-safe`, in violation of the requirements of POSIX.1. This was fixed in glibc 2.24.

Architecture-specific details
On `sparc` and `sparc64`, `execv()` is provided as a system call by the kernel (with the prototype shown above) for compatibility with SunOS. This function is *not* employed by the `execv()` wrapper function on those architectures.

SEE ALSO
`sh(1)`, `execve(2)`, `execveat(2)`, `fork(2)`, `ptrace(2)`, `fexecve(3)`, `system(3)`, `environ(7)`

Linux man-pages 6.17 2025-05-17 `exec(3)`


```
saicharan_reddi@SAICHARAN x + v
GNU nano 8.7.1 process.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    char command[100];

    printf("Enter command: ");
    scanf("%s", command);

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
    }
    else if (pid == 0)
    {
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());

        execlp(command, command, NULL);

        printf("Exec failed!\n");
    }
    else
    {
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        wait(NULL);
    }

    return 0;
}
```

Report:

A child process was created successfully using `fork()`, and the user command was executed using `exec()`. The experiment successfully demonstrated Linux process creation and execution using system calls.